ENTORNOS DE DESARROLLO

UD2_ACT6
SCRUM VS KANBAN

FRANCELLA ROJAS C

Contenido

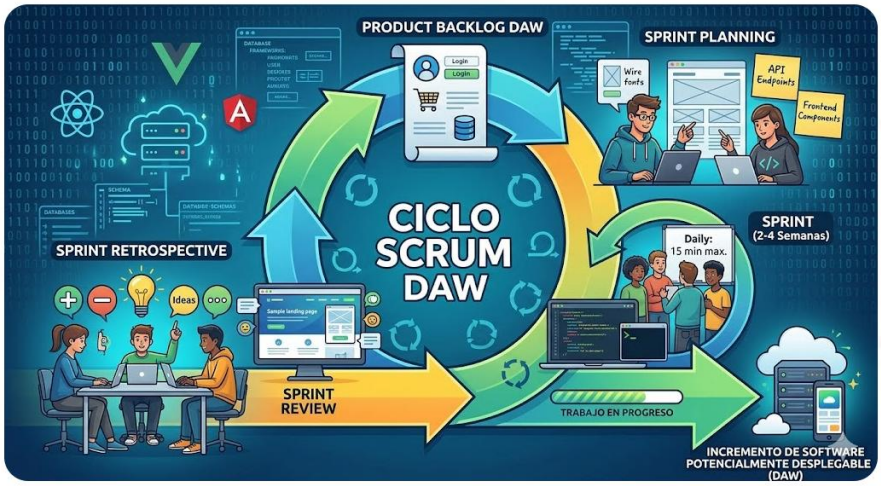
INTRODUCCIÓN 3
Metodología SCRUM 4
¿Cómo funciona un Sprint? 4
Sprint Planning: 4
Daily Scrum: 4
Sprint Review: 4
Retrospectiva: 4
Los roles en Scrum 4
Product Owner: 4
Scrum Master: 4
Equipo de Desarrollo: 5
¿Por qué usarlo? 5
Metodología KANBAN 5
Visualizar el trabajo 5
Backlog: 5
To Do: 5
In Progress: 5
Testing/Review: 5
Done: 5
El WIP (Work In Progress) 6
Gestión del flujo 6
¿Por qué usar Kanban en lugar de Scrum? 6
SCRUM VS KANBAN 6
Implementación en Github Proyects 7
GITHUB-KANBAN 7
GITHUB-SCRUM 12
Conclusiones 14
Bibliografía 15

INTRODUCCIÓN

En el desarrollo de software actual, ya no basta con saber programar; lo más difícil suele ser organizarse bien para no perderse entre tantos archivos, errores y cambios de última hora. Por eso, en este trabajo de Entornos de Desarrollo, vamos a analizar las dos metodologías que más se usan en las empresas reales: Scrum y Kanban.

La idea no es solo soltar la teoría de qué es cada una, sino entender sus diferencias y ver cómo podemos llevarlas a la práctica usando GitHub Projects. Al final, el objetivo es aprender a gestionar un proyecto de forma ágil, aprovechando que GitHub nos da todas las herramientas para que el flujo de trabajo sea mucho más visual y ordenado, tanto si trabajamos solos como si lo hacemos en equipo.

Metodología SCRUM



Scrum es una de las metodologías "ágiles" más usadas porque se basa en avanzar poco a poco, pero de forma muy segura. En lugar de planificar todo el proyecto de principio a fin (que luego siempre cambia algo), lo que hacemos es dividir el trabajo en ciclos cortos llamados Sprints.

¿Cómo funciona un Sprint?

Un Sprint suele durar entre 2 y 4 semanas. La idea es que, al acabar ese tiempo, tengamos una parte del programa que ya funcione y se pueda enseñar. No vale decir "estoy en ello", tiene que haber algo terminado.

Para que esto funcione, seguimos estos pasos:

Sprint Planning: Al principio del ciclo, el equipo decide qué tareas del "Backlog" (la lista total de cosas por hacer) se van a meter en este Sprint.

Daily Scrum: Es una reunión rapidísima de unos 15 minutos cada día. Solo decimos qué hicimos ayer, qué vamos a hacer hoy y si tenemos algún problema que nos bloquee.

Sprint Review: Al final del Sprint, enseñamos lo que hemos programado para recibir feedback.

Retrospectiva: El equipo se junta para hablar de qué ha ido bien y qué podemos mejorar en el siguiente Sprint para no cometer los mismos fallos.

Los roles en Scrum

Para que el equipo no sea un caos, nos repartimos así:

Product Owner: Es la persona que sabe qué necesita el cliente. Se encarga de priorizar las tareas más importantes.

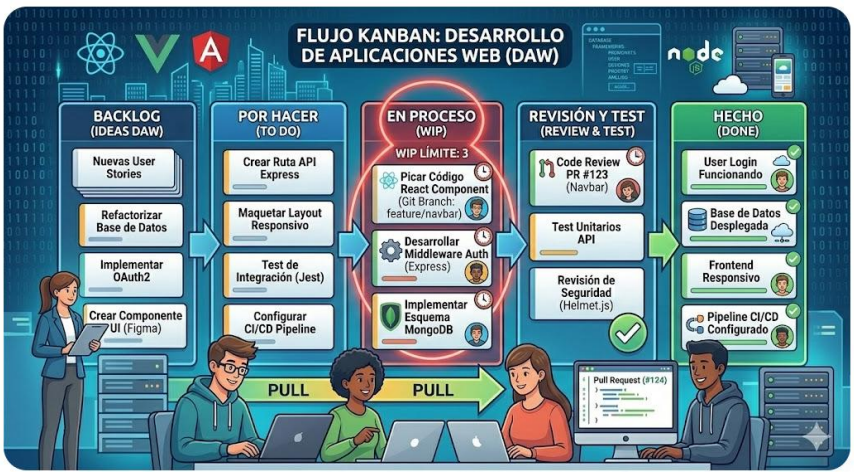
Scrum Master: No es un jefe, es más como un "facilitador". Su trabajo es que el equipo siga las reglas de Scrum y quitar los obstáculos que nos encontremos.

Equipo de Desarrollo: Los que picamos código y sacamos el trabajo adelante.

¿Por qué usarlo?

Lo mejor de Scrum es que, si a mitad del proyecto el cliente quiere cambiar algo, no pasa nada. Como trabajamos por Sprints, podemos meter ese cambio en el siguiente ciclo sin que se desmonte todo lo que ya hemos hecho. Es muy flexible y te obliga a estar siempre concentrado en terminar tareas reales.

Metodología KANBAN



Si Scrum era trabajar por "partidos" (Sprints), Kanban es como una cadena de montaje infinita. No hay fechas de inicio ni de fin para un grupo de tareas; el trabajo simplemente fluye por el tablero de izquierda a derecha.

Se basa en tres reglas muy simples:

Visualizar el trabajo

La herramienta principal es el Tablero Kanban. En DAW, esto es clave para no volverse loco con los commits. Dividimos el tablero en columnas de estado, normalmente:

Backlog: Ideas o tareas que hay que hacer algún día.

To Do: Tareas seleccionadas para empezar ya.

In Progress: Lo que estás programando en este mismo momento.

Testing/Review: Cuando has hecho el Pull Request y esperas a que alguien vea si tu código no rompe nada.

Done: Tarea terminada y desplegada.

El WIP (Work In Progress)

Esta es la parte que más le gusta a los profes de Entornos. Kanban dice que hay que limitar el trabajo en curso. Si decides que tu límite de "In Progress" es 2, no puedes empezar una tercera tarea hasta que termines una de las que tienes abiertas.

Esto sirve para evitar el "multitasking" (que al final es no terminar nada) y detectar dónde está el cuello de botella (por ejemplo, si hay 10 tareas en Review y nadie las revisa).

Gestión del flujo

A diferencia de Scrum, aquí no hay roles como el "Scrum Master". El equipo se autogestiona. Si entra una tarea urgente (un bug crítico en la web, por ejemplo), se mete en el tablero y se empieza en cuanto haya un hueco libre en el WIP. Es mucho más flexible para proyectos que cambian cada día.

¿Por qué usar Kanban en lugar de Scrum?

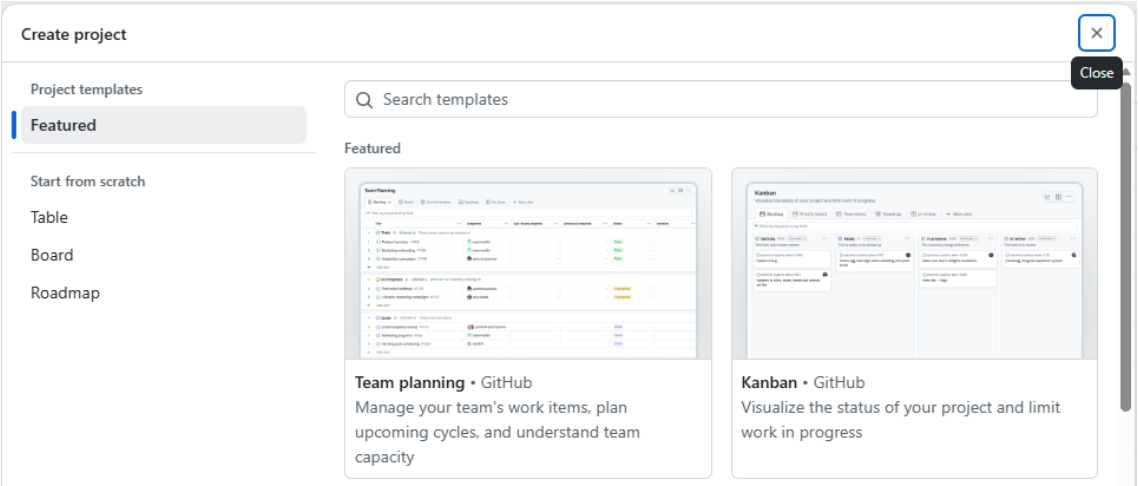
- Es menos estresante porque no tienes la presión de que "se acaba el Sprint".
- Es ideal para mantenimiento de apps o proyectos donde las tareas entran de forma constante.
- Es mucho más fácil de implementar si eres un equipo pequeño o incluso si trabajas tú sola.

SCRUM VS KANBAN

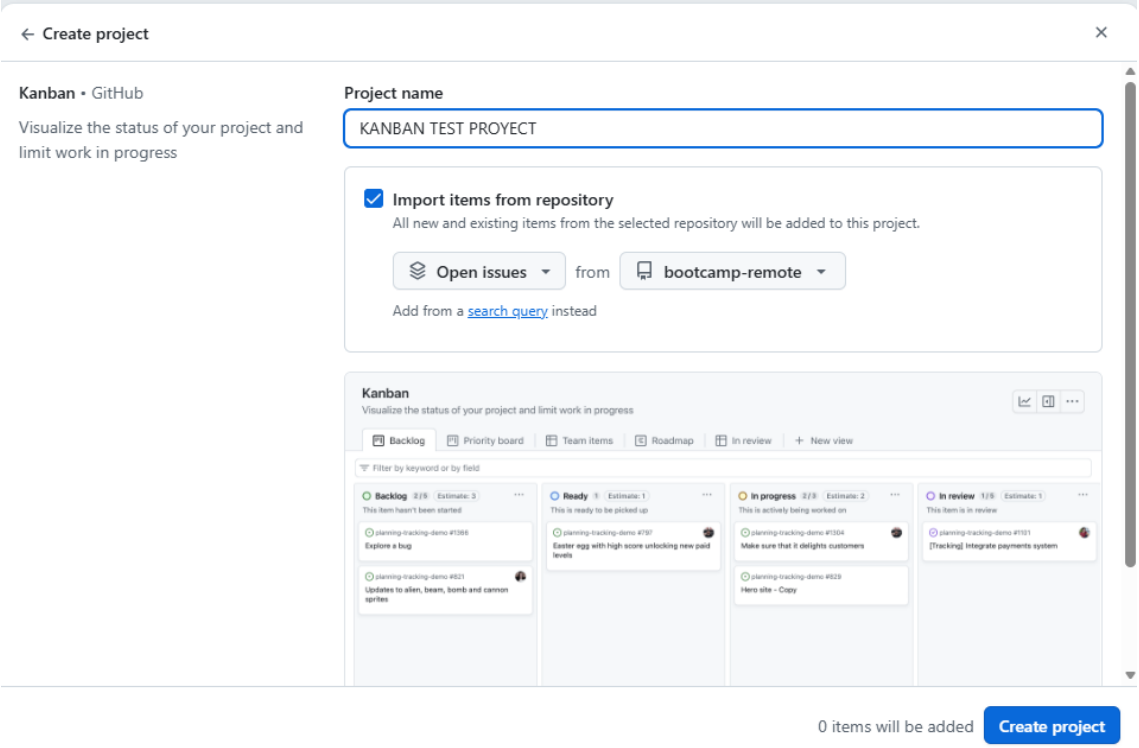
Característica	SCRUM	KANBAN
Enfoque	Basado en el tiempo (Sprints).	Basado en el flujo visual (Continuo).
Roles	Definidos (Scrum Master, Product Owner).	No hay roles fijos.
Cambios	No se aceptan durante el Sprint.	Se pueden añadir en cualquier momento.
Planificación	Al inicio de cada Sprint.	Continua, según se liberan huecos.
Métricas	Velocidad (puntos por Sprint).	Tiempo de ciclo (cuánto tarda una tarea).
Uso en DAW	Proyectos con entregas cerradas (hitos).	Mantenimiento, corrección de bugs o micro-servicios.

Implementación en Github Proyects

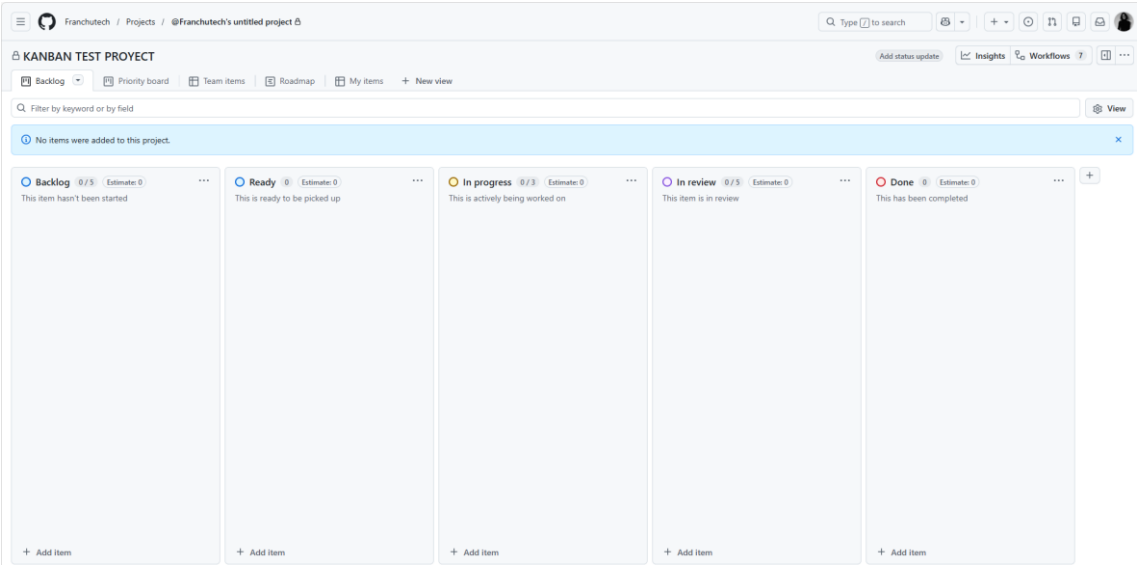
GITHUB-KANBAN



La plataforma de GITHUB tiene la opción de crear proyectos y darnos a elegir opciones compatibles con cada método, en este caso tiene un template específico para KANBAN el cual utilizaremos.



Le doy un nombre, y decido bajo que repositorio estaré trabajando, en este caso utilicé un repositorio viejo que tengo de un bootcamp.



Como se observa en la imagen la realidad es que es demasiado intuitivo y lo poco que no sepas lo buscas en google o IA y ya lo tendrías resuelto.

Como se observa en la captura, he establecido un límite de 3 tareas simultáneas en la columna de proceso para respetar la filosofía Kanban y no saturar el desarrollo.

Básicamente tiene:

Priority board: Es la vista principal del proyecto. Aquí es donde gestiono el flujo de trabajo real. Las tareas se mueven de izquierda a derecha (desde que se detecta una necesidad en el Backlog hasta que se completa en Done). Es fundamental para detectar si hay atascos en la columna de Review.

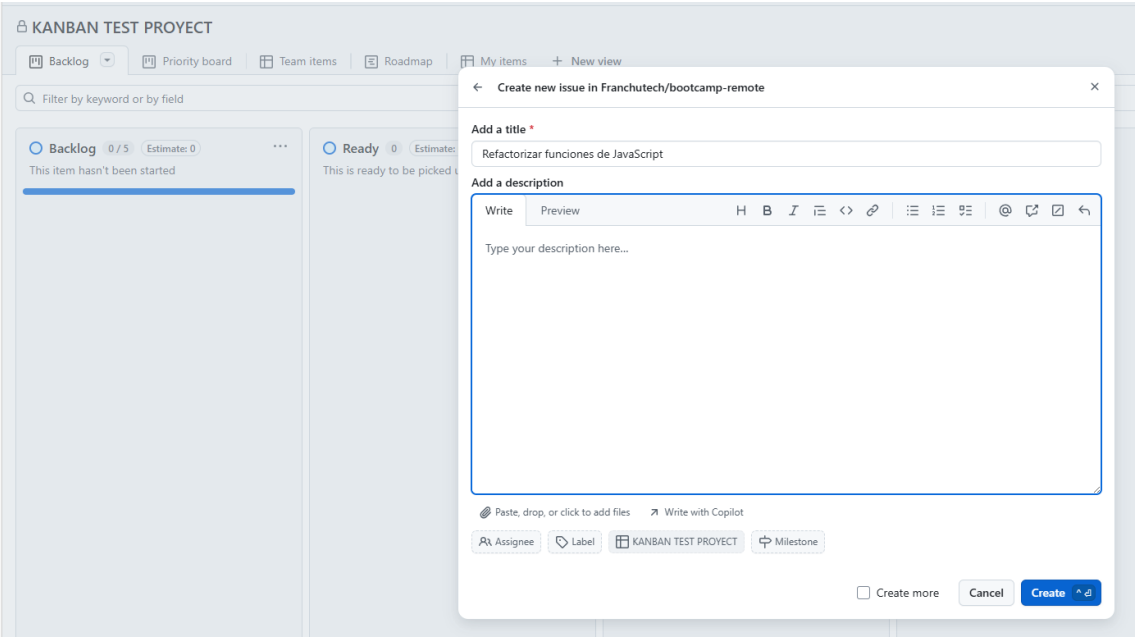
Backlog: Es el listado donde guardo todas las ideas, errores a corregir o nuevas funcionalidades para la web que aún no he empezado a desarrollar. Me permite tener el tablero principal limpio y centrado solo en lo que estoy haciendo ahora.

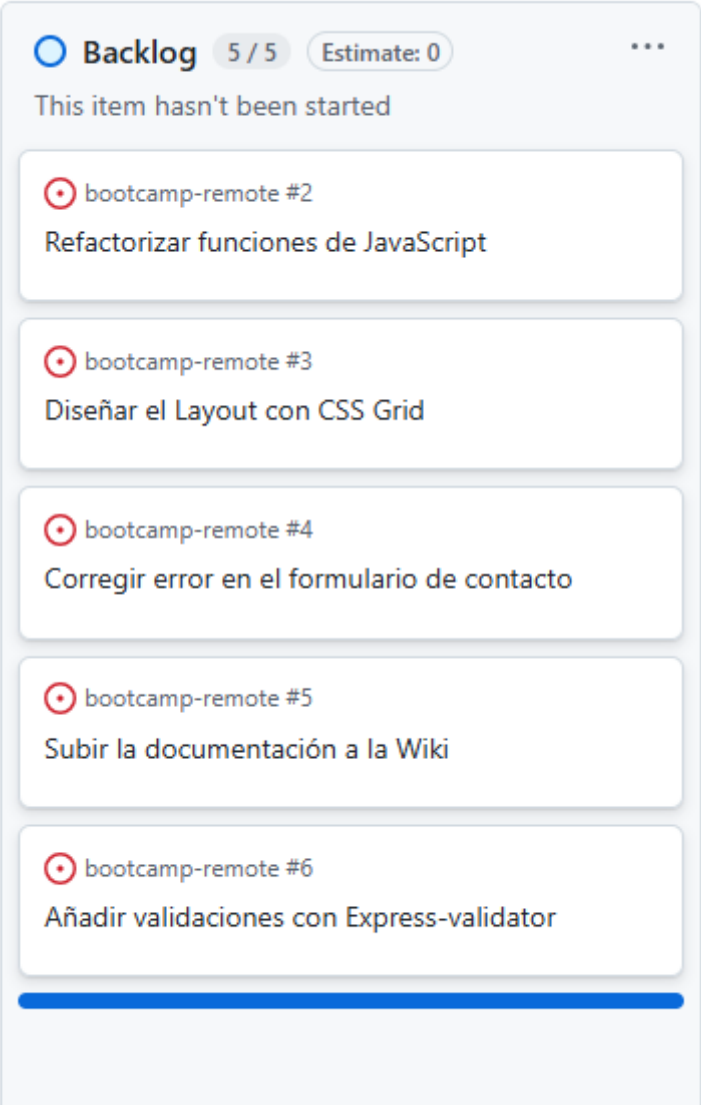
Team items: Esta pestaña me permite organizar el trabajo por personas. En nuestro caso, me sirve para ver de un vistazo qué tareas tengo yo y qué tareas tiene mi pareja, asegurándonos de que cada una sabe qué parte del código le toca tocar.

Roadmap: Me da una visión temporal del proyecto. Aunque en Kanban el flujo es continuo, esta vista me ayuda a prever cuánto tiempo nos están llevando las funcionalidades más grandes y si el ritmo de desarrollo es el adecuado.

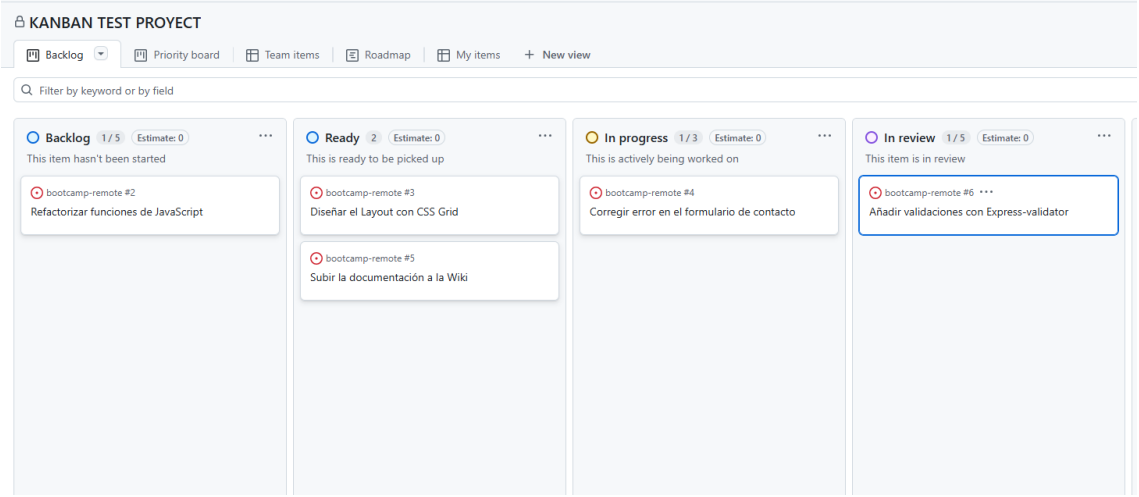
My items: Es mi panel de control personal. Aquí solo veo las tareas que tengo asignadas directamente, lo que me ayuda a concentrarme en mi código sin distraerme con el volumen total de tareas del proyecto.

Con el fin de trastear un poco creo algunos ISSUES

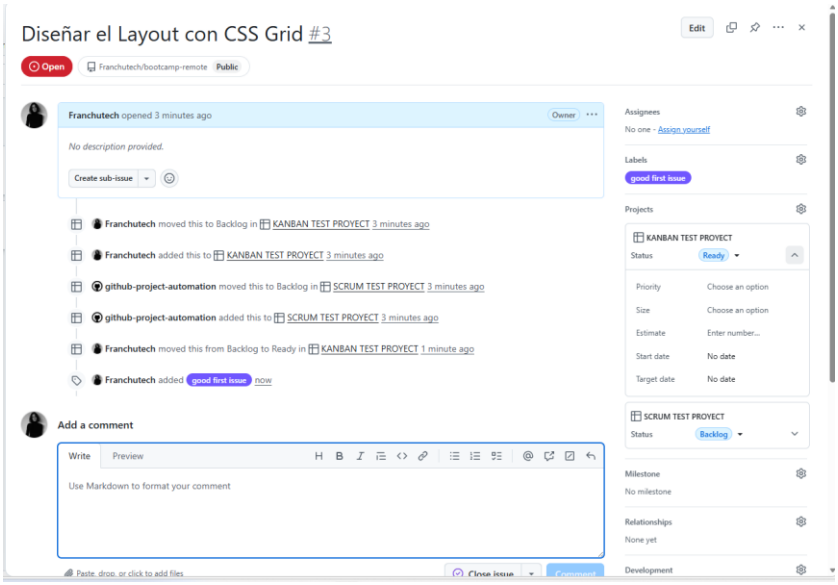

KANBAN TEST PROYECT
Backlog
Priority board
Team items
Roadmap
My items
New view
Filter by keyword or by field
Backlog 0 / 5 Estimate: 0
This item hasn't been started
Ready 0
Create new issue in Franchutech/bootcamp-remote
Add a title *
Refactorizar funciones de JavaScript
Add a description
Write
Preview
Type your description here...
Paste, drop, or click to add files
Write with Copilot
Assignee
Label
KANBAN TEST PROYECT
Milestone
Create more
Cancel
Create


Backlog 5 / 5 Estimate: 0
This item hasn't been started
bootcamp-remote #2
Refactorizar funciones de JavaScript
bootcamp-remote #3
Diseñar el Layout con CSS Grid
bootcamp-remote #4
Corregir error en el formulario de contacto
bootcamp-remote #5
Subir la documentación a la Wiki
bootcamp-remote #6
Añadir validaciones con Express-validator

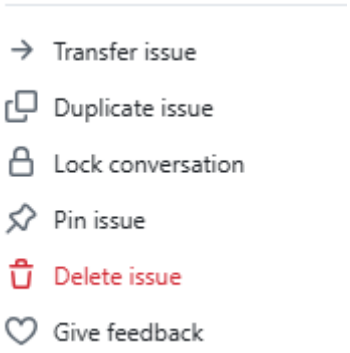
Las tareas pueden arrastrarse según su status



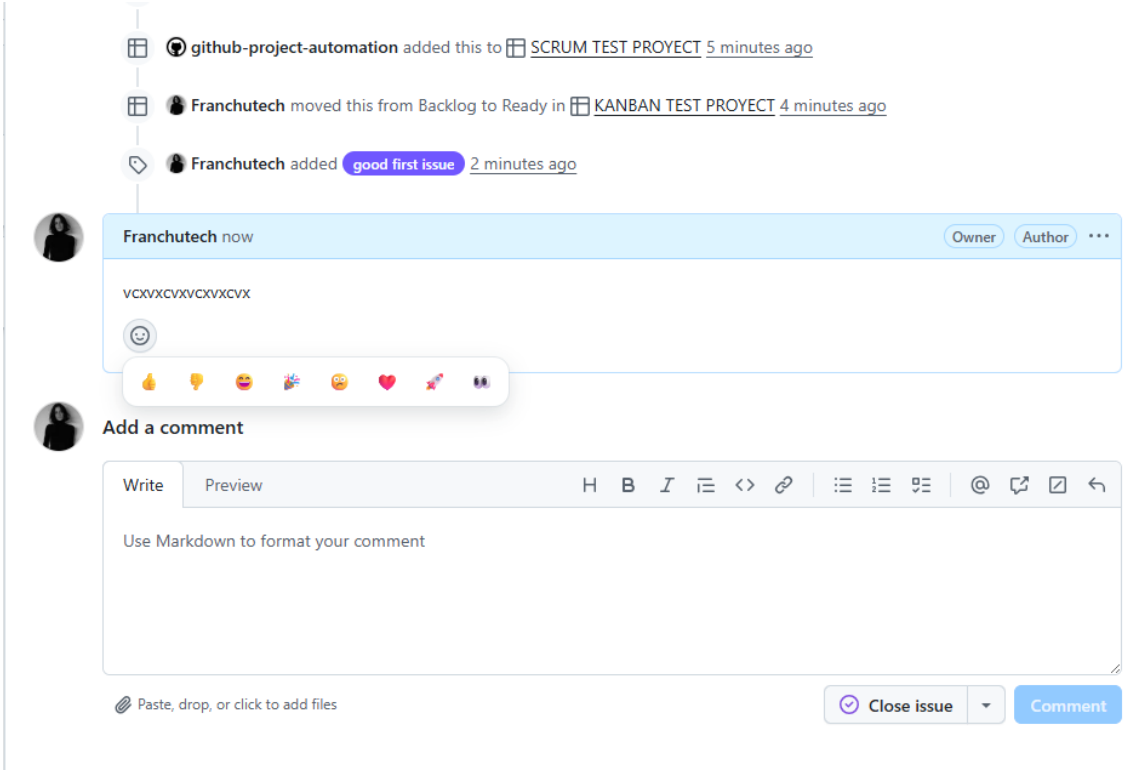
Luego tocando cada tarea realmente puedes editar muchas cosas, asignarla, definirle colores según lo que quieras, ya como modificar a gusto personal y organizativo de cada quién



Tiene además algunos botones interesantes



Incluso tiene para escribir comentarios o reaccionar a ellos:



En general puedes organizar tu trabajo como quieras. (o bueno un proyecto como el equipo lo necesite, es bastante versátil)

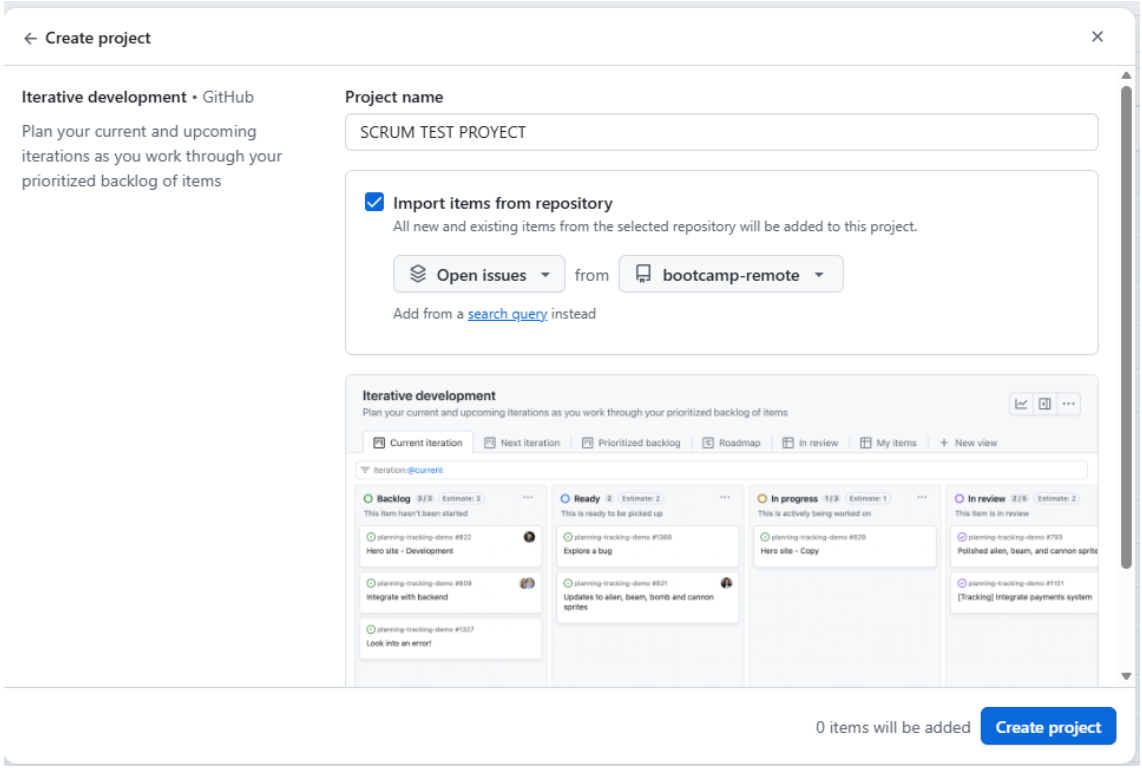
Y pues puedes asignarlas a alguien en particular entre otros:

Luego vemos cómo sale la modificación de que fue asignado

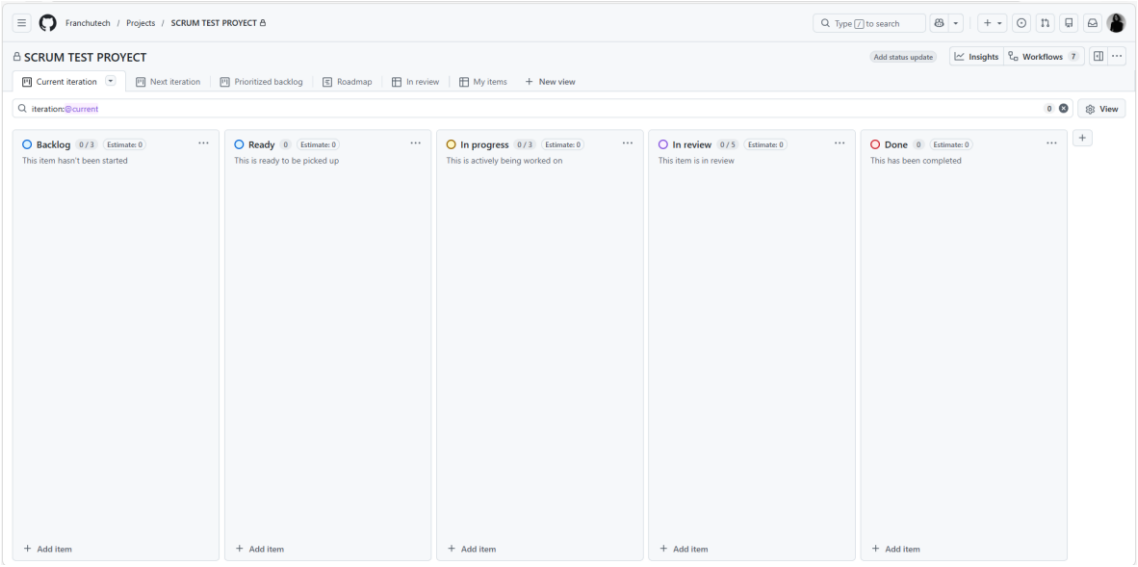
Y funciona en esencia igual para el template de Scrum.

GITHUB-SCRUM

Para este otro hago exactamente los mismos pasos y en este caso utilizo el template recomendado por el profesor ITERATIVE DEVELOPMENT que es el más parecido a Scrum según nos mencionó en clase.



Y básicamente es similar, pero trabaja a base de Iteraciones, que serían los famosos Sprints.



Este diría que es un poco menos intuitivo al menos para mí, sin embargo, como dije antes un poco de google e IA y lo tienes resuelto realmente.

Sin embargo, para ampliar como el anterior detallaré un poco cada pestaña:

Current iteration: Es la pestaña más importante de Scrum. Aquí solo veo las tareas que he metido en el Sprint actual. Me sirve para no agobiarme con el resto del proyecto y centrarme solo en lo que tengo que terminar antes de que acabe la iteración.

Next iteration: Esta es pura planificación. Aquí voy moviendo las tareas que sé que no me da tiempo a hacer ahora, pero que serán la prioridad absoluta en el siguiente Sprint. Es como tener el trabajo ya organizado antes de empezar.

Prioritized backlog: A diferencia del Kanban, aquí el backlog suele estar ordenado por importancia. Pongo arriba del todo lo que más valor aporta a la web (como el login o la base de datos) para asegurarme de que se asigne a una iteración lo antes posible.

Roadmap: En Scrum esta vista es clave, porque me permite ver visualmente en un calendario cuánto dura cada iteración. Así puedo controlar que no se solapen y ver si llegamos a tiempo a la entrega final del proyecto de DAW.

In review: He dejado esta pestaña específica para controlar qué partes del código están pendientes de que mi pareja o algún compañero las revise. En Scrum, nada se da por terminado hasta que pasa el filtro de calidad.

My items: Mi rincón personal. Filtra el tablero para enseñarme solo mis tareas de este Sprint. Así sé exactamente qué código tengo que picar yo hoy sin liarme con las tareas de los demás.

Conclusiones

En este trabajo he comprobado que la organización es tan importante como el código. Mientras que Scrum me ha servido para trabajar con la disciplina de los Sprints y las entregas por ciclos cerrados, Kanban me ha resultado mucho más flexible y visual para gestionar el flujo diario.

La clave no es elegir una metodología y seguirla a ciegas, sino saber cuál encaja mejor en cada momento del desarrollo. Usar GitHub Projects me ha permitido llevar estas teorías a la práctica de forma real, conectando mis tareas directamente con el repositorio. Al final, dominar estas herramientas me permite trabajar de forma más ágil, evitar atascos y asegurar que cada línea de código que escribo tiene un sentido dentro del proyecto.

Bibliografía

Scrum Guide (Guía oficial): La Guía de Scrum: Las reglas del juego. Escrito por Ken Schwaber y Jeff Sutherland. Es el documento base para entender los roles y eventos de Scrum.

PDF DE LECTURA: 2020-Scrum-Guide-Spanish-European.pdf

Documentación de GitHub: Manual de uso de GitHub Projects y gestión de Issues. Github dispone de muchos documentos para sus distintas funcionalidades.

https://docs.github.com/es/issues

Atlassian Agile Coach: Metodologías Ágiles: Diferencias entre el flujo Kanban y el marco Scrum. Una referencia que me resulto interesante leer.

https://www.atlassian.com/es/agile/kanban/kanban-vs-scrum

Kanban University: Principios y prácticas del método Kanban para equipos de software. Lectura que me pareció muy interactiva al respecto y saque algunas ideas de ella.

https://kanban.university/wp-content/uploads/2024/03/The-Official-Kanban-Guide_Spanish.pdf

Microsoft Learn: Introducción al desarrollo iterativo y marcos de trabajo ágiles. Muy enfocado a cómo se gestionan los proyectos en entornos de desarrollo modernos.

https://learn.microsoft.com/es-es/devops/plan/what-is-agile-development